

SmartPOS

Supermarket POS & Inventory System

MVP Functional & Technical Specification — Revised

Version 2.0 · June 2026

This revision corrects gaps identified in the original draft: branch/multi-store handling, split-payment support, sales returns, a unified stock-movement audit log, and an explicit tax model. Items changed from the original draft are marked with a Revision Note.

Table of Contents

Table of Contents	2
1. Project Overview	5
1.1 System Name	5
1.2 Type	5
1.3 Target Users	5
1.4 Scope Decision: Single-Branch vs. Multi-Branch.....	5
2. Core Modules (MVP)	5
2.1 Authentication & User Management	5
Features.....	5
Roles	5
2.2 Dashboard	6
Dashboard Widgets	6
2.3 Product Management.....	6
Fields.....	6
Operations	6
2.4 Category Management.....	7
Features.....	7
2.5 Supplier Management	7
Fields.....	7
Features.....	7
2.6 Customer Management.....	7
Fields.....	7
Features.....	7
2.7 POS Sales Module.....	8
Main Screen Layout	8
Features.....	8
Payments.....	8
Receipt	8
2.8 Inventory Management.....	9
Features.....	9
2.9 Purchase Management	9
Fields.....	9
Workflow	9
2.10 Sales Returns	9
Features.....	9

2.11 Expense Management	10
Fields	10
2.12 Tax Model	10
3. Database Design	11
3.1 branches	11
3.2 users	11
3.3 products	11
3.4 categories	12
3.5 customers	12
3.6 suppliers	13
3.7 sales & payments	13
sales	13
payments (revision: new table)	13
sale_items	14
3.8 stock_movements	14
3.9 sales_returns & return_items	14
sales_returns	15
return_items	15
3.10 purchases & purchase_items	15
purchases	15
purchase_items	16
3.11 tax_rates	16
3.12 expenses	16
4. Reports Module	17
4.1 Sales Reports	17
4.2 Inventory Reports	17
4.3 Financial Reports	17
4.4 Cashier Reports	17
5. Suggested Tech Stack	17
5.1 Frontend	17
5.2 Backend — Recommended: Laravel	17
5.3 Database	18
5.4 Authentication	18
5.5 Barcode	18
5.6 Receipt Printing	18
6. API Endpoints	19

6.1 Authentication	19
6.2 Products	19
6.3 Sales	19
6.4 Sales Returns	19
6.5 Purchases.....	19
6.6 Stock Movements	20
7. MVP Development Roadmap.....	21
Phase 1 (Week 1) — Foundations.....	21
Phase 2 (Week 2) — Catalog.....	21
Phase 3 (Week 3) — Point of Sale	21
Phase 4 (Week 4) — Inventory & Purchasing	21
Phase 5 (Week 5) — Reporting.....	21
8. Future Enhancements	21
9. Summary of Changes from Original Draft.....	23

1. Project Overview

1.1 System Name

SmartPOS Supermarket System

1.2 Type

Web-based Point of Sale (POS) and Inventory Management System.

1.3 Target Users

- Cashier
- Store Manager
- Administrator
- Inventory Officer

1.4 Scope Decision: Single-Branch vs. Multi-Branch

Revision Note: the original draft listed stock transfers between branches as an MVP feature, but had no branch/store field anywhere in the schema, and listed multi-branch support as a future enhancement. This contradiction is resolved below.

This MVP is scoped for a single store location. A nullable `branch_id` field is included in the schema from the start (see Section 3) so that multi-branch support can be added later as a data migration rather than a schema redesign. Inter-branch stock transfer as a working feature is moved to Phase 2 / Future Enhancements; it is not part of the MVP feature set.

2. Core Modules (MVP)

2.1 Authentication & User Management

Features

- Login / Logout
- Role-based access control
- Password reset
- User profile

Roles

Role	Permissions
Admin	Full access
Manager	Reports + Inventory
Cashier	Sales only
Stock Officer	Inventory only

2.2 Dashboard

Displays:

- Today's Sales
- Total Transactions
- Products in Stock
- Low Stock Alerts
- Top Selling Products
- Recent Transactions

Dashboard Widgets

Widget	Example Value
Today's Sales	TZS 2,450,000
Transactions	145
Products	2,350
Low Stock	12

Additional widgets: Sales Chart, Top Products, Recent Sales.

2.3 Product Management

Fields

- Product Name
- Barcode
- SKU
- Category
- Brand
- Cost Price
- Selling Price
- Tax Class (see Section 2.12)
- Unit
- Stock Quantity
- Minimum Stock
- Expiry Date
- Supplier
- Image

Operations

- Add product
- Edit product

- Delete product
- Search product
- Barcode generation
- Import via Excel

2.4 Category Management

Examples: Beverages, Groceries, Electronics, Cosmetics, Stationery, Household.

Features

- Add category
- Edit category
- Delete category

2.5 Supplier Management

Fields

- Supplier Name
- Contact Person
- Phone
- Email
- Address
- TIN Number

Features

- Add supplier
- Update supplier
- View purchases

2.6 Customer Management

Optional for walk-in customers.

Fields

- Customer Name
- Phone Number
- Address
- Email
- Loyalty Points

Features

- Customer history
- Loyalty points

- Credit purchases

2.7 POS Sales Module

Main Screen Layout

Product	Qty	Price	Total
Rice	2	3,500	7,000
Sugar	1	4,500	4,500
Subtotal		11,500	
Discount		500	
Tax		1,000	
Grand Total		12,000	

Revision Note: "Cash Received" and "Change" are shown on screen during checkout, but since this MVP supports split payments, the underlying data model uses a separate payments table rather than single amount_received / change_amount fields (see Section 3.7).

Features

- Barcode scanning
- Search product
- Add multiple products to cart
- Quantity adjustment
- Discount
- Tax calculation
- Suspend sale
- Resume sale
- Hold sale

Payments

Supported payment methods:

- Cash
- Mobile Money
- Card
- Bank Transfer
- Split payments (e.g. part cash, part mobile money)

Receipt

Printed receipt includes:

- Store Name, Address, Phone
- Receipt No. and Date
- Cashier name
- Line items (Qty, Price, Total)

- Grand Total
- Payment method(s) used
- “Thank You” footer message

2.8 Inventory Management

Features

- Stock In — Purchase entry (Supplier, Quantity, Date)
- Stock Out — Damages, expired goods, returns
- Adjustments — Manual increase/decrease of stock with a required reason code
- Transfers — Move stock between branches (Phase 2, depends on multi-branch support)

Revision Note: every stock-affecting action above (sale, purchase, adjustment, return, damage write-off) must write a row to the new stock_movements table (Section 3.8) so stock discrepancies can always be traced back to a cause.

2.9 Purchase Management

Fields

- Supplier
- Invoice Number
- Product
- Quantity
- Cost Price
- Date

Workflow

1. Create Purchase Order
2. Approve
3. Receive Goods
4. Update Stock

2.10 Sales Returns

Revision Note: the original draft described this as a feature but had no corresponding database tables. A sales_returns / return_items structure is added in Section 3.9, with an explicit refund method field.

Features

- Return products (full or partial)
- Refund customer (cash, store credit, or original payment method reversal)
- Exchange products

2.11 Expense Management

Examples: Rent, Electricity, Internet, Transport, Salary.

Fields

- Expense Type
- Amount
- Date
- Description

2.12 Tax Model

Revision Note: the original draft used a single flat tax field on the product and a single tax line in the cart. This does not support per-item tax rates, tax-exempt goods, or a clear choice between tax-inclusive and tax-exclusive pricing — all of which are common in real supermarket catalogs (e.g. VAT-exempt staple foods). The MVP now defines this explicitly rather than leaving it implicit.

MVP tax rules:

- Each product is assigned a `tax_class` (e.g. STANDARD, EXEMPT, ZERO_RATED) rather than a raw percentage.
- Tax classes map to rates in a small, admin-editable `tax_rates` table, decoupling rate changes from product records.
- Product prices are stored and entered as tax-inclusive by default, matching standard supermarket shelf pricing; the receipt breaks out the tax portion for transparency.
- Each `sale_item` stores the tax rate applied at time of sale, so historical receipts remain accurate even if rates change later.

3. Database Design

Tables added or materially changed from the original draft are marked with a Revision Note. Foreign keys are denoted with the suffix `_id`.

3.1 branches

Revision Note: new table. Supports the single-branch MVP cleanly while making multi-branch a data change, not a schema change. All operational tables below carry a nullable `branch_id` referencing this table.

Field	Type	Notes
id	PK	
name	string	
address	string	
phone	string	
is_main_branch	boolean	True for the single MVP location
created_at	timestamp	

3.2 users

Field	Type	Notes
id	PK	
name	string	
email	string	Unique
phone	string	
password	string	Hashed, never stored in plain text
role	enum	admin / manager / cashier / stock_officer
branch_id	FK → branches.id	Nullable; revision addition
status	enum	active / suspended
created_at	timestamp	

3.3 products

Field	Type	Notes
id	PK	
barcode	string	Unique
sku	string	Unique

Field	Type	Notes
name	string	
category_id	FK → categories.id	
supplier_id	FK → suppliers.id	
cost_price	decimal	
selling_price	decimal	Tax-inclusive (see Section 2.12)
tax_class_id	FK → tax_rates.id	Revision: replaces flat 'tax' field
unit	string	e.g. pcs, kg, litre
stock_quantity	integer	
minimum_stock	integer	Triggers low-stock alert
expiry_date	date	Nullable
image	string	File path / URL
status	enum	active / discontinued
created_at	timestamp	

3.4 categories

Field	Type	Notes
id	PK	
name	string	
description	string	Nullable

3.5 customers

Field	Type	Notes
id	PK	
name	string	
phone	string	
email	string	Nullable
address	string	Nullable
points	integer	Loyalty points balance

3.6 suppliers

Field	Type	Notes
id	PK	
name	string	
phone	string	
email	string	Nullable
address	string	Nullable
tin_number	string	Nullable

3.7 sales & payments

Revision Note: the original draft stored `payment_method`, `amount_received`, and `change_amount` as flat columns on the sales table. This cannot represent a split payment (e.g. half cash, half mobile money). A separate payments table is introduced; sales keeps only the order-level totals.

sales

Field	Type	Notes
id	PK	
invoice_no	string	Unique
customer_id	FK → customers.id	Nullable for walk-in
cashier_id	FK → users.id	
branch_id	FK → branches.id	Revision addition
subtotal	decimal	
discount	decimal	
tax_total	decimal	
grand_total	decimal	
status	enum	completed / suspended / refunded / partially_refunded — revision addition
created_at	timestamp	

payments (revision: new table)

Field	Type	Notes
id	PK	
sale_id	FK → sales.id	
method	enum	cash / mobile_money / card / bank

Field	Type	Notes
amount	decimal	Allows multiple rows per sale for split payments
amount_received	decimal	For cash only
change_given	decimal	For cash only
reference_no	string	Nullable; mobile money / card transaction ref

sale_items

Field	Type	Notes
id	PK	
sale_id	FK → sales.id	
product_id	FK → products.id	
quantity	integer	
price	decimal	Unit price at time of sale
tax_rate_applied	decimal	Revision addition; preserves historical accuracy
total	decimal	

3.8 stock_movements

Revision Note: new table. The original draft had no single source of truth for why stock levels change. Every stock-affecting event (sale, purchase receipt, adjustment, return, damage, transfer) writes one row here, giving a full audit trail.

Field	Type	Notes
id	PK	
product_id	FK → products.id	
branch_id	FK → branches.id	
change_qty	integer	Positive for stock in, negative for stock out
reason	enum	sale / purchase / adjustment / return / damage / expiry / transfer
reference_type	string	e.g. 'sale', 'purchase', 'sales_return'
reference_id	integer	ID of the related sale/purchase/return record
user_id	FK → users.id	Who performed/triggered the action
notes	string	Nullable, free text
created_at	timestamp	

3.9 sales_returns & return_items

Revision Note: new tables, backing the Sales Returns feature described in Section 2.10, which had no schema in the original draft.

sales_returns

Field	Type	Notes
id	PK	
sale_id	FK → sales.id	
processed_by	FK → users.id	
refund_method	enum	cash / store_credit / original_payment_reversal
refund_amount	decimal	
reason	string	
created_at	timestamp	

return_items

Field	Type	Notes
id	PK	
sales_return_id	FK → sales_returns.id	
sale_item_id	FK → sale_items.id	
quantity	integer	
restocked	boolean	Whether the item was returned to sellable stock

3.10 purchases & purchase_items

purchases

Field	Type	Notes
id	PK	
supplier_id	FK → suppliers.id	
branch_id	FK → branches.id	Revision addition
invoice_no	string	
status	enum	draft / approved / received — revision addition, reflects the workflow in Section 2.9
total_amount	decimal	
date	date	

purchase_items

Field	Type	Notes
id	PK	
purchase_id	FK → purchases.id	
product_id	FK → products.id	
quantity	integer	
cost_price	decimal	

3.11 tax_rates

Revision Note: new table, backing the tax model defined in Section 2.12.

Field	Type	Notes
id	PK	
name	string	e.g. 'Standard VAT', 'Exempt', 'Zero-Rated'
rate_percent	decimal	0 for exempt/zero-rated
is_active	boolean	

3.12 expenses

Field	Type	Notes
id	PK	
expense_type	string	
amount	decimal	
description	string	Nullable
branch_id	FK → branches.id	Revision addition
date	date	

4. Reports Module

4.1 Sales Reports

- Daily Sales
- Weekly Sales
- Monthly Sales
- Annual Sales

4.2 Inventory Reports

- Current Stock
- Low Stock
- Out of Stock
- Expiry Report

Revision Note: with the stock_movements table in place, an additional Stock Movement / Audit report becomes possible at low extra cost — useful for investigating discrepancies. Recommended as a Phase 4 addition.

4.3 Financial Reports

- Profit / Loss
- Expenses
- Revenue

4.4 Cashier Reports

- Sales per cashier

5. Suggested Tech Stack

Revision Note: the original draft listed two alternative backend frameworks (Node/Express and Laravel) without committing to one. They imply different auth systems, ORMs, and project structures, so one should be chosen before development starts. Laravel is recommended below for the reasons given; either is workable.

5.1 Frontend

- React.js
- Next.js
- Tailwind CSS
- ShadCN UI

5.2 Backend — Recommended: Laravel

Laravel is recommended over a Node.js/Express stack for this project because:

- Built-in authentication (Laravel Sanctum) removes the need to hand-roll JWT handling.
- Eloquent ORM and migrations map cleanly onto the relational schema in Section 3.
- Built-in queueing and scheduling are useful later for things like automated low-stock alerts or expiry notifications.

If the team's existing expertise is in JavaScript/TypeScript, Node.js + Express + Prisma is a reasonable alternative — but should be chosen explicitly rather than left open.

5.3 Database

- MySQL

5.4 Authentication

- Laravel Sanctum (if Laravel backend)
- JWT (if Node.js backend)

5.5 Barcode

- JsBarcode — generates barcode images for display/printing

Revision Note: JsBarcode only generates barcode images; it does not control physical label printers. If the store needs to print its own product labels (rather than only scanning supplier barcodes), a thermal/label printer integration (e.g. ZPL or EPL command support) needs separate scoping — this is a meaningfully different piece of work.

5.6 Receipt Printing

- react-to-print

6. API Endpoints

6.1 Authentication

Method	Endpoint
POST	/api/login
POST	/api/logout
POST	/api/register

6.2 Products

Method	Endpoint
GET	/api/products
POST	/api/products
PUT	/api/products/:id
DELETE	/api/products/:id

6.3 Sales

Method	Endpoint
POST	/api/sales
GET	/api/sales
GET	/api/sales/:id

6.4 Sales Returns

Revision Note: new endpoints, since this is now a fully scoped MVP feature with a backing schema (Section 3.9).

Method	Endpoint
POST	/api/sales/:id/returns
GET	/api/sales-returns

6.5 Purchases

Method	Endpoint
POST	/api/purchases
GET	/api/purchases

Method	Endpoint
POST	/api/purchases/:id/approve
POST	/api/purchases/:id/receive

Revision Note: approve/receive endpoints added to match the multi-step purchase workflow already described in Section 2.9.

6.6 Stock Movements

Revision Note: new read endpoint, exposing the audit trail described in Section 3.8.

Method	Endpoint
GET	/api/stock-movements
GET	/api/products/:id/stock-movements

7. MVP Development Roadmap

Revision Note: schema additions (branches, payments, stock_movements, sales_returns, tax_rates) are front-loaded into Phase 1 and 2 below, since changing the database design after Phase 3 (POS Sales) is built is significantly more expensive than including it from the start.

Phase 1 (Week 1) — Foundations

- Authentication
- Dashboard
- User Roles
- Full database schema migration, including branches, tax_rates, payments, and stock_movements tables (revision: moved earlier)

Phase 2 (Week 2) — Catalog

- Product Management
- Categories
- Suppliers

Phase 3 (Week 3) — Point of Sale

- POS Sales Screen
- Receipt Printing
- Barcode Scanning
- Split payment support (revision: explicit deliverable)

Phase 4 (Week 4) — Inventory & Purchasing

- Inventory
- Purchases
- Stock Management
- Sales Returns (revision: moved up from being unscheduled in the original draft)

Phase 5 (Week 5) — Reporting

- Reports
- Expenses

8. Future Enhancements

- Multi-branch support (stock transfers between branches, branch-level reporting)
- Online orders
- E-commerce integration
- Customer loyalty program

- SMS notifications
- Mobile app
- AI sales forecasting
- Offline synchronization
- Accounting integration
- QR payment support

Revision Note: offline synchronization is worth re-evaluating for inclusion in the MVP rather than deferring it, depending on the reliability of internet connectivity at the deployment site. A cashier unable to process sales during an outage is a serious operational risk; even a minimal 'queue and sync later' fallback may be worth the added cost.

9. Summary of Changes from Original Draft

Area	Change
Branches	Added branches table and branch_id across operational tables; multi-branch transfer feature explicitly deferred to Phase 2/Future, resolving a contradiction in the original scope.
Payments	Replaced flat payment_method/amount_received/change_amount columns on sales with a separate payments table to properly support split payments.
Stock Audit Trail	Added stock_movements table so every stock change (sale, purchase, adjustment, return, damage) is traceable to a cause and an actor.
Sales Returns	Added sales_returns and return_items tables and corresponding API endpoints; the feature existed in the original draft text but had no backing schema.
Tax Model	Replaced a single flat tax field with a tax_rates table and per-product tax_class, supporting exempt/zero-rated goods and preserving historical tax rates on past sales.
Purchases	Added a status field (draft / approved / received) to match the already-described approval workflow, plus matching API endpoints.
Tech Stack	Resolved the open choice between Node.js and Laravel; Laravel recommended with stated rationale.
Barcode Printing	Flagged that JsBarcode covers image generation only, not physical label printer integration, which needs separate scoping if required.
Roadmap	Schema work for the above additions moved into Phase 1 rather than left implicit, to avoid costly rework after POS Sales (Phase 3) is built.